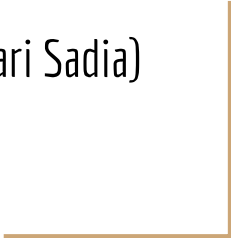




Data Structures

(Multi-Dimensional Array
and Linked List)

Naeem Ahmed
(Slides adapted from Mushtari Sadia)



Introduction

Faculty Name: Naeem Ahmed (NMA)

Room: UB80601

Consultation Hours:

Sunday: 9:30 AM - 11:00 AM

Monday: 11:00 AM-12:30 PM

Wednesday: 11:00 AM-2:00 PM

Slide link: <http://tiny.cc/cse220week2NMA>

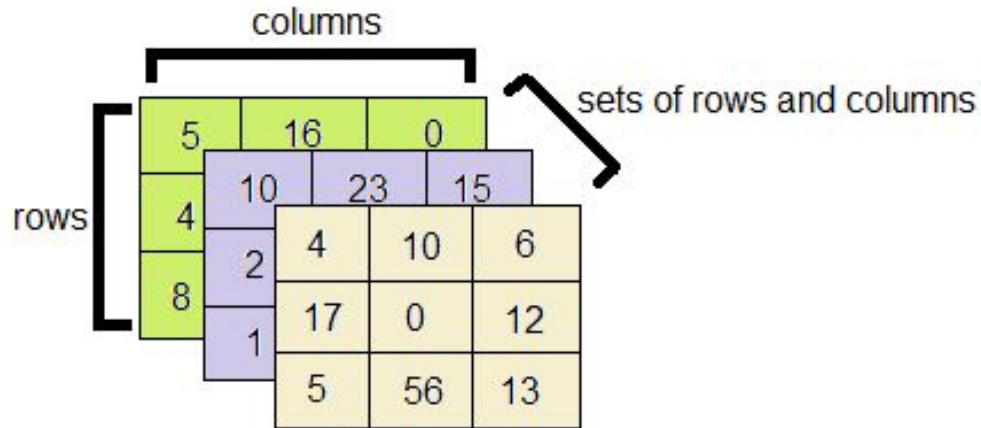
Multi-Dimensional Array

2-D Array

1	1	1	1	1	1
1	0	1	1	0	1
1	1	1	1	1	1
1	0	1	1	0	1
1	0	0	0	0	1
1	1	1	1	1	1

Multi-Dimensional Array

3-D Array



How are multi-dimensional arrays stored?

There are two standards here:

1. Increasing the final dimension's index first and progressively move to earlier dimensions, this is called the **Row-Major Ordering**. For example, Java and C take this approach.
2. Increasing the starting dimension's index first and progressively move to later dimensions, this is called the **Column-Major Ordering**. For example, FORTRAN takes this approach.

How are multi-dimensional arrays stored?

The following diagram shows the difference between these two approaches for a 2D matrix of 3×3 dimensions. (By the way, row is the vertical and column is the horizontal axis in 2D. You should already know that. Here Dimension 0 represents the row and Dimension 1 represents the column.)

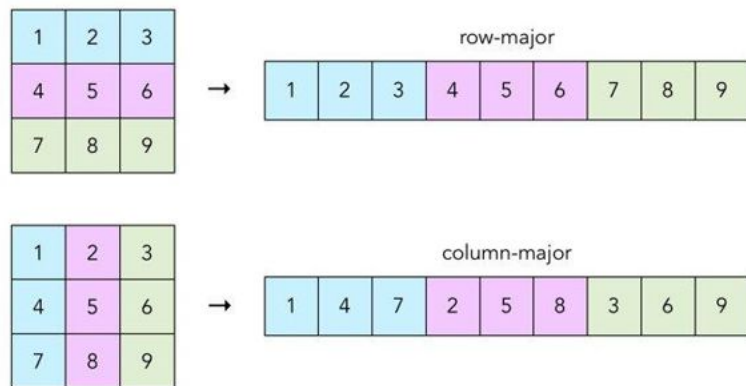


Figure 1: Two Different Ways of Storing Multidimensional Arrays in a Linear Format

How are multi-dimensional arrays stored?

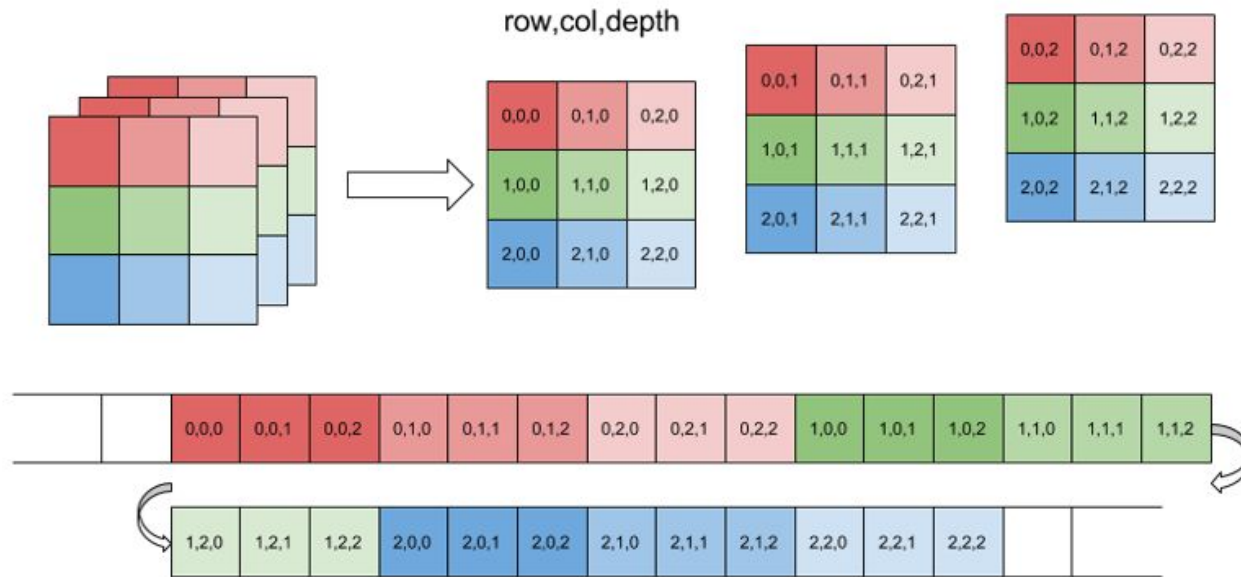


Figure 2: Row Major Ordering of Indexes of a 3D Array

Multidimensional Array Index -> Linear Index

imagine you have a 4D array with dimensions $M \times N \times O \times P$, named *box*

element at the index you are interested is the *box[w][x][y][z]*.

$$\text{Linear Index} = w \times (N \times O \times P) + x \times (O \times P) + y \times P + z$$

Multidimensional Array Index $\rightarrow$ Linear Index

imagine you have a 4D array with dimensions $M \times N \times O \times P$, named *box*

element at the index you are interested is the *box[w][x][y][z]*.

$$\text{Linear Index} = \underline{w \times (N \times O \times P)} + x \times (O \times P) + y \times P + z$$

Multidimensional Array Index -> Linear Index

imagine you have a 4D array with dimensions $M \times N \times O \times P$, named *box*

element at the index you are interested is the *box* $[w][x][y][z]$.

$$\text{Linear Index} = w \times (N \times O \times P) + \underline{x \times (O \times P)} + y \times P + z$$

Multidimensional Array Index $\rightarrow$ Linear Index

imagine you have a 4D array with dimensions $M \times N \times O \times P$, named *box*

element at the index you are interested is the *box*[w][x][y][z].

$$\text{Linear Index} = w \times (N \times O \times P) + x \times (O \times P) + \underline{y \times P} + z$$

Multidimensional Array Index $\rightarrow$ Linear Index

imagine you have a 4D array with dimensions $M \times N \times O \times P$, named *box*

element at the index you are interested is the *box*[*w*][*x*][*y*][*z*].

$$\text{Linear Index} = w \times (N \times O \times P) + x \times (O \times P) + y \times P + \underline{z}$$

Multidimensional Array Index -> Linear Index

imagine you have a 4D array with dimensions 4x3x5x6, named *box*

element at the index you are interested is the `box[2][1][4][3]`

Linear Index = ??

Linear Index -> Multidimensional Array Index

Imagine you have a 3D array with dimensions 4x4x8.

What are the multidimensional indexes of the element stored at location 111?

Linear Index = 111 -> `box[x][y][z]`

Find x,y,z

Linear Index -> Multidimensional Array Index

Imagine you have a 3D array with dimensions 4x4x8.

What are the multidimensional indexes of the element stored at location 111?

$$\text{Linear Index} = x * (4*8) + y*8 + z$$

$$= \mathbf{111}$$

Linear Index -> Multidimensional Array Index

Imagine you have a 3D array with dimensions 4x4x8.

What are the multidimensional indexes of the element stored at location 111?

$$\begin{aligned} \text{Linear Index} &= x * (4*8) + y*8 + z && \text{where } 0 \leq x \leq 3, 0 \leq y \leq 3 \text{ and } 0 \leq z \leq 7 \\ &= \mathbf{111} \end{aligned}$$

Linear Index -> Multidimensional Array Index

Imagine you have a 3D array with dimensions 4x4x8.

What are the multidimensional indexes of the element stored at location 111?

$$\begin{aligned}\text{Linear Index} &= x * (4*8) + y*8 + z \\ &= 111\end{aligned}$$

where $0 \leq x \leq 3$, $0 \leq y \leq 3$ and $0 \leq z \leq 7$

Remember that indexing starts from 0 in each dimension!

Linear Index -> Multidimensional Array Index

Imagine you have a 3D array with dimensions 4x4x8.

What are the multidimensional indexes of the element stored at location 111?

$$\begin{aligned}\text{Linear Index} &= x * (4*8) + y*8 + z \\ &= 111\end{aligned}$$

where $0 \leq x \leq 3$, $0 \leq y \leq 3$ and $0 \leq z \leq 7$

Remember that indexing starts from 0 in each dimension!

$$4 \times 8 =$$

			0	0	3
3	2	1	1	1	
	-	0			
		1	1		
	-		0		
		1	1	1	
	-		9	6	
			1	5	

Quotient

$x=3$

Remainder

Linear Index -> Multidimensional Array Index

Following the process we get,

$$X = 111 // (4 * 8) = 3 \text{ and } 111 \% (4 * 8) = 15$$

$$Y = 15 // 8 = 1 \text{ and } 15 \% 8 = 7$$

$$Z = 7$$

So, the dimension of linear index 111 is [3][1][7].

Linear Index -> Multidimensional Array Index

Practice

Find the dimension of 96, 107, and 60 of the linear index.

Array Initialization with NumPy

▼ Learning To Use Numpy

```
✓ [1] 1 import numpy as np  
0s
```

```
✓ [5] 1 arr = np.array([1, 2, 3, 4, 5])  
0s 2  
3 print(arr)
```

```
[1 2 3 4 5]
```

```
✓ [2] 1 array = np.zeros(5)  
0s 2 print(array)
```

```
[0. 0. 0. 0. 0.]
```

Array Initialization with NumPy

✓
0s

```
[3] 1 array[2] = 5  
    2 print(array)
```

[0. 0. 5. 0. 0.]

```
[4] 1 for i in range(len(array)):  
    2 |   array[i] = i+1  
    3  
    4 print(array)
```

[1. 2. 3. 4. 5.]

Array Initialization with NumPy

✓ 0s 1 print(array[1])

2.0

✗ 0s [7] 1 print(array[11])

```
-----  
IndexError                                Traceback (most recent call last)  
<ipython-input-7-92562eb7d277> in <cell line: 1>()  
----> 1 print(array[11])
```

IndexError: index 11 is out of bounds for axis 0 with size 5

[SEARCH STACK OVERFLOW](#)

✓ 0s [8] 1 print(array[0])

1.0

✗ 0s [10] 1 print(array[5])

```
-----  
IndexError                                Traceback (most recent call last)  
<ipython-input-10-7de47b3bb65c> in <cell line: 1>()  
----> 1 print(array[5])
```

IndexError: index 5 is out of bounds for axis 0 with size 5

Multi-Dimensional Array Initialization with NumPy

▼ Multidimensional Array Initialization

```
[ ] 1 # Create a 2-D array containing two arrays with the values 1,2,3 and 4,5,6:  
2 arr = np.array([[1, 2, 3], [4, 5, 6]])  
3  
4 print(arr)
```

```
[[1 2 3]  
 [4 5 6]]
```

```
[12] 1 #Create a 2D array with the dimensions 2x3  
2 arr = np.zeros((2,3))  
3 print(arr)
```

```
[[0. 0. 0.]  
 [0. 0. 0.]]
```

Multi-Dimensional Array Initialization with NumPy

```
✓ [13] 1  #Create a 3D array with the dimensions 2x3x5  
0s      2  
        3  arr = np.zeros((2,3,5))  
        4  print(arr)
```

```
[[[0. 0. 0. 0. 0.]  
  [0. 0. 0. 0. 0.]  
  [0. 0. 0. 0. 0.]]  
  
[[0. 0. 0. 0. 0.]  
 [0. 0. 0. 0. 0.]  
 [0. 0. 0. 0. 0.]]]
```

Multi-Dimensional Array Initialization with NumPy

```
✓ [13] 1  #Create a 3D array with the dimensions 2x3x5  
0s      2  
        3  arr = np.zeros((2,3,5))  
        4  print(arr)
```

```
[[[0. 0. 0. 0. 0.]  
  [0. 0. 0. 0. 0.]  
  [0. 0. 0. 0. 0.]
```

```
[[[0. 0. 0. 0. 0.]  
  [0. 0. 0. 0. 0.]  
  [0. 0. 0. 0. 0.]
```

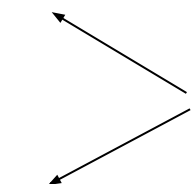
Two lists

Multi-Dimensional Array Initialization with NumPy

```
✓ [13] 1 #Create a 3D array with the dimensions 2x3x5  
0s     2  
       3 arr = np.zeros((2,3,5))  
       4 print(arr)
```

```
[[[0. 0. 0. 0. 0.]  
  [0. 0. 0. 0. 0.]  
  [0. 0. 0. 0. 0.]
```

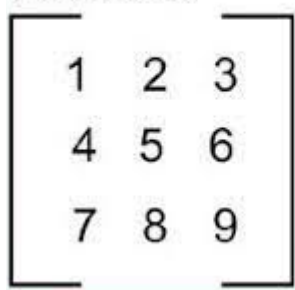
```
[[[0. 0. 0. 0. 0.]  
  [0. 0. 0. 0. 0.]  
  [0. 0. 0. 0. 0.]
```



Two lists **having**
three lists each

Multidimensional Array implementation

- A matrix can be presented as a 2d array.



1	2	3
4	5	6
7	8	9

[[1,2,3],
[4,5,6],
[7,8,9]]

How to Access 2D Array Elements

```
import numpy as np
```

```
x = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12]])  
print(x)
```

```
[[ 1  2  3]  
 [ 4  5  6]  
 [ 7  8  9]  
 [10 11 12]]
```

How to Access 2D Array Elements

```
for i in range(len(x)):
    for j in range(len(x[0])):
        print(x[i][j], " ")
    print()
```

len(x) gives the 1st dimension (number of rows)

len(x[0]) gives the 2nd dimension (number of columns)

Matrix Multiplication

$$X = \begin{bmatrix} 12 & 7 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

dimension= 3 X 3

$$Y = \begin{bmatrix} 5 & 8 & 1 & 2 \\ 6 & 7 & 3 & 0 \\ 5 & 8 & 9 & 1 \end{bmatrix}$$

dimension= 3 X 4

Find out XY

Matrix Multiplication

$$X = \begin{bmatrix} 12 & 7 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

dimension= 3 X 3

$$Y = \begin{bmatrix} 5 & 8 & 1 & 2 \\ 6 & 7 & 3 & 0 \\ 5 & 8 & 9 & 1 \end{bmatrix}$$

dimension= 3 X 4

Dimension of XY = ?

Matrix Multiplication

$$X = \begin{bmatrix} 12 & 7 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

dimension= 3 X 3

$$Y = \begin{bmatrix} 5 & 8 & 1 & 2 \\ 6 & 7 & 3 & 0 \\ 5 & 8 & 9 & 1 \end{bmatrix}$$

dimension= 3 X 4

Dimension of $XY = 3 \times 4$

Matrix Multiplication

$$\begin{bmatrix} 12 & 7 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \times \begin{bmatrix} 5 & 8 & 1 & 2 \\ 6 & 7 & 3 & 0 \\ 5 & 8 & 9 & 1 \end{bmatrix} = \begin{bmatrix} 117 & 169 & 60 & 27 \\ 80 & 115 & 73 & 14 \\ 128 & 184 & 112 & 23 \end{bmatrix}$$

**$12*5 + 7*6 + 3*5 = 117$: from 1st row of
Matrix 1 and 1st column of Matrix 2**

Matrix Multiplication

$$\begin{bmatrix} 12 & 7 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \times \begin{bmatrix} 5 & 8 & 1 & 2 \\ 6 & 7 & 3 & 0 \\ 5 & 8 & 9 & 1 \end{bmatrix} = \begin{bmatrix} 117 & 169 & 60 & 27 \\ 80 & 115 & 73 & 14 \\ 128 & 184 & 112 & 23 \end{bmatrix}$$

from 1st row of Matrix 1 and 2nd
column of Matrix 2

Matrix Multiplication

$$\begin{bmatrix} 12 & 7 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \times \begin{bmatrix} 5 & 8 & 1 & 2 \\ 6 & 7 & 3 & 0 \\ 5 & 8 & 9 & 1 \end{bmatrix} = \begin{bmatrix} 117 & 169 & 60 & 27 \\ 80 & 115 & 73 & 14 \\ 128 & 184 & 112 & 23 \end{bmatrix}$$

from 1st row of Matrix 1 and 3rd column
of Matrix 2

Matrix Multiplication

$$\begin{bmatrix} 12 & 7 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \times \begin{bmatrix} 5 & 8 & 1 & 2 \\ 6 & 7 & 3 & 0 \\ 5 & 8 & 9 & 1 \end{bmatrix} = \begin{bmatrix} 117 & 169 & 60 & 27 \\ 80 & 115 & 73 & 14 \\ 128 & 184 & 112 & 23 \end{bmatrix}$$

from 1st row of Matrix 1 and 4th column
of Matrix 2

Matrix Multiplication

In General:

To multiply an $m \times n$ matrix by an $n \times p$ matrix, the n s must be the same, and the result is an $m \times p$ matrix.

$$m \times n \times n \times p \rightarrow m \times p$$

```
if array_1.shape[1] != array_2.shape[0]:  
    print("Matrix multiplication not possible, wrong dimensions")
```


Matrix Multiplication

$$m \times n \times n \times p \rightarrow m \times p$$

```
m = array_1.shape[0]
n = array_1.shape[1]
p = array_2.shape[1]

result = np.zeros((m,p))
```

Matrix Multiplication

m = 3
p = 4

```
for i in range(m):  
    for j in range(p):  
        sum=0  
        for k in range(n):  
            sum += array_1[i][k]*array_2[k][j]  
        result[i][j] = sum
```

$$\begin{bmatrix} 1 & 2 & 1 \\ 0 & 1 & 0 \\ 2 & 3 & 4 \end{bmatrix} \times \begin{bmatrix} 2 & 5 & 1 & 1 \\ 6 & 7 & 4 & 2 \\ 1 & 8 & 3 & 4 \end{bmatrix} = \begin{bmatrix} \bullet \end{bmatrix}$$

Matrix Multiplication

m = 3
p = 4

```
for i in range(m):  
    for j in range(p):  
        sum=0  
        for k in range(n):  
            sum += array_1[i][k]*array_2[k][j]  
        result[i][j] = sum
```

$$\begin{bmatrix} 1 & 2 & 1 \\ 0 & 1 & 0 \\ 2 & 3 & 4 \end{bmatrix} \times \begin{bmatrix} 2 & 5 & 1 & 1 \\ 6 & 7 & 4 & 2 \\ 1 & 8 & 3 & 4 \end{bmatrix}$$

=

$$\begin{bmatrix} \bullet & \bullet \\ & \bullet & \bullet \\ & & \bullet & \bullet \end{bmatrix}$$

Matrix Multiplication

m = 3
p = 4

```
for i in range(m):  
    for j in range(p):  
        sum=0  
        for k in range(n):  
            sum += array_1[i][k]*array_2[k][j]  
        result[i][j] = sum
```

$$\begin{bmatrix} 1 & 2 & 1 \\ 0 & 1 & 0 \\ 2 & 3 & 4 \end{bmatrix} \times \begin{bmatrix} 2 & 5 & 1 & 1 \\ 6 & 7 & 4 & 2 \\ 1 & 8 & 3 & 4 \end{bmatrix}$$

=

$$\begin{bmatrix} \bullet & \bullet & \bullet \\ & & \\ & & \end{bmatrix}$$

Matrix Multiplication

m = 3
p = 4

```
for i in range(m):  
    for j in range(p):  
        sum=0  
        for k in range(n):  
            sum += array_1[i][k]*array_2[k][j]  
        result[i][j] = sum
```

$$\begin{bmatrix} 1 & 2 & 1 \\ 0 & 1 & 0 \\ 2 & 3 & 4 \end{bmatrix} \times \begin{bmatrix} 2 & 5 & 1 & 1 \\ 6 & 7 & 4 & 2 \\ 1 & 8 & 3 & 4 \end{bmatrix}$$

=

$$\begin{bmatrix} \bullet & \bullet & \bullet & \bullet \\ & & & \\ & & & \end{bmatrix}$$

Matrix Multiplication

m = 3
p = 4

```
for i in range(m):  
    for j in range(p):  
        sum=0  
        for k in range(n):  
            sum += array_1[i][k]*array_2[k][j]  
        result[i][j] = sum
```

$$\begin{bmatrix} 1 & 2 & 1 \\ 0 & 1 & 0 \\ 2 & 3 & 4 \end{bmatrix} \times \begin{bmatrix} 2 & 5 & 1 & 1 \\ 6 & 7 & 4 & 2 \\ 1 & 8 & 3 & 4 \end{bmatrix}$$

=

$$\begin{bmatrix} \bullet & \bullet & \bullet & \bullet \\ \bullet & \bullet & \bullet & \bullet \end{bmatrix}$$

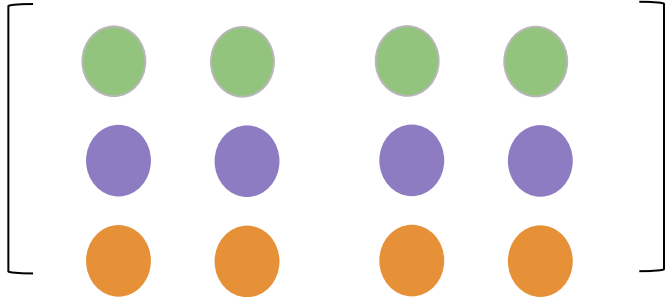
Matrix Multiplication

m = 3
p = 4

```
for i in range(m):  
    for j in range(p):  
        sum=0  
        for k in range(n):  
            sum += array_1[i][k]*array_2[k][j]  
        result[i][j] = sum
```

$$\begin{bmatrix} 1 & 2 & 1 \\ 0 & 1 & 0 \\ 2 & 3 & 4 \end{bmatrix} \times \begin{bmatrix} 2 & 5 & 1 & 1 \\ 6 & 7 & 4 & 2 \\ 1 & 8 & 3 & 4 \end{bmatrix}$$

=



Matrix Multiplication

m = 3
p = 4
n = 3

```
for i in range(m):  
    for j in range(p):  
        sum=0  
        for k in range(n):  
            sum += array_1[i][k]*array_2[k][j]  
        result[i][j] = sum
```

$$\begin{bmatrix} 1 & 2 & 1 \\ 0 & 1 & 0 \\ 2 & 3 & 4 \end{bmatrix} \times \begin{bmatrix} 2 & 5 & 1 & 1 \\ 6 & 7 & 4 & 2 \\ 1 & 8 & 3 & 4 \end{bmatrix}$$

= $\begin{bmatrix} \text{●} & = 1 \times 2 + 2 \times 6 + 1 \times 1 \end{bmatrix}$

Matrix Multiplication

```
1 def matrix_multiplication(array_1,array_2):
2
3     if array_1.shape[1]!=array_2.shape[0]:
4         print("Matrix multiplication not possible, wrong dimensions")
5         return
6
7     m = array_1.shape[0]
8     n = array_1.shape[1]
9     p = array_2.shape[1]
10
11     result = np.zeros((m,p))
12
13     for i in range(m):
14         for j in range(p):
15             sum=0
16             for k in range(n):
17                 sum += array_1[i][k]*array_2[k][j]
18             result[i][j] = sum
19
20     return result
21
```

Matrix Multiplication

```
1 def matrix_multiplication(array_1,array_2):
2
3     if array_1.shape[1]!=array_2.shape[0]:
4         print("Matrix multiplication not possible, wrong dimensions")
5         return
6
7     m = array_1.shape[0]
8     n = array_1.shape[1]
9     p = array_2.shape[1]
10
11     result = np.zeros((m,p))
12
13     for i in range(m):
14         for j in range(p):
15             sum=0
16             for k in range(n):
17                 sum += array_1[i][k]*array_2[k][j]
18             result[i][j] = sum
19
20     return result
21
```

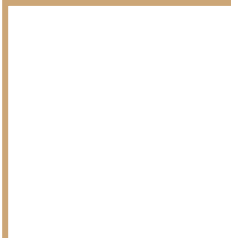
```
1 array_1 = np.array([[1,2,3],[4,5,6]])
2 array_2 = np.array([[7,8],[9,10],[11,12]])
```

```
1 print(array_1.shape)
2 print(array_2.shape)
```

```
(2, 3)
(3, 2)
```

```
1 print(matrix_multiplication(array_1,array_2))
```

```
[[ 58.  64.]
 [139. 154.]]
```



Linked List



Limitations of Array

- **Fixed capacity** : Once created, an array cannot be resized. The only way to "resize" is to create a larger new array, copy the elements from the original array into the new one, and then change the reference to the new one.

Limitations of Array

- **Fixed capacity** : Once created, an array cannot be resized. The only way to "resize" is to create a larger new array, copy the elements from the original array into the new one, and then change the reference to the new one.
- **Shifting elements in insert** : Since we do not allow gaps in the array, inserting a new element requires that we shift all the subsequent elements right to create a hole where the new element is placed. In the worst case, we "disturb" every slot in the array when we insert it at the beginning of the array!

Limitations of Array

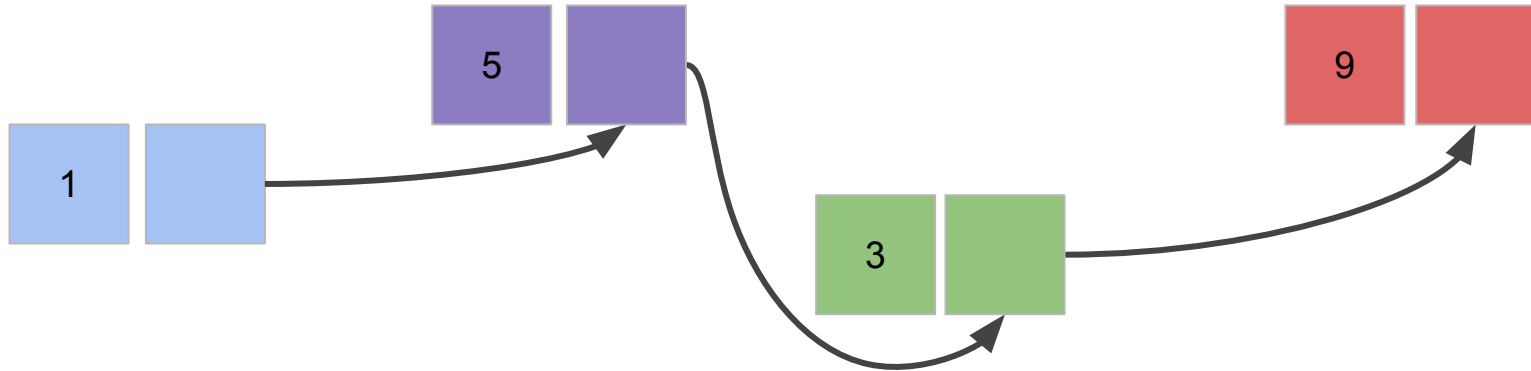
- **Fixed capacity** : Once created, an array cannot be resized. The only way to "resize" is to create a larger new array, copy the elements from the original array into the new one, and then change the reference to the new one.
- **Shifting elements in insert** : Since we do not allow gaps in the array, inserting a new element requires that we shift all the subsequent elements right to create a hole where the new element is placed. In the worst case, we "disturb" every slot in the array when we insert it at the beginning of the array!
- **Shifting elements in removal** : Removing may also require shifting to plug the hole left by the removed element. If the ordering of the elements does not matter, we can avoid shifting by replacing the removed element with the element in the last slot (and then treating the last slot as empty).

Array vs. Linked List

Array
in memory



Linked list
in memory



Resizing - Array

Array
Size 5

arr



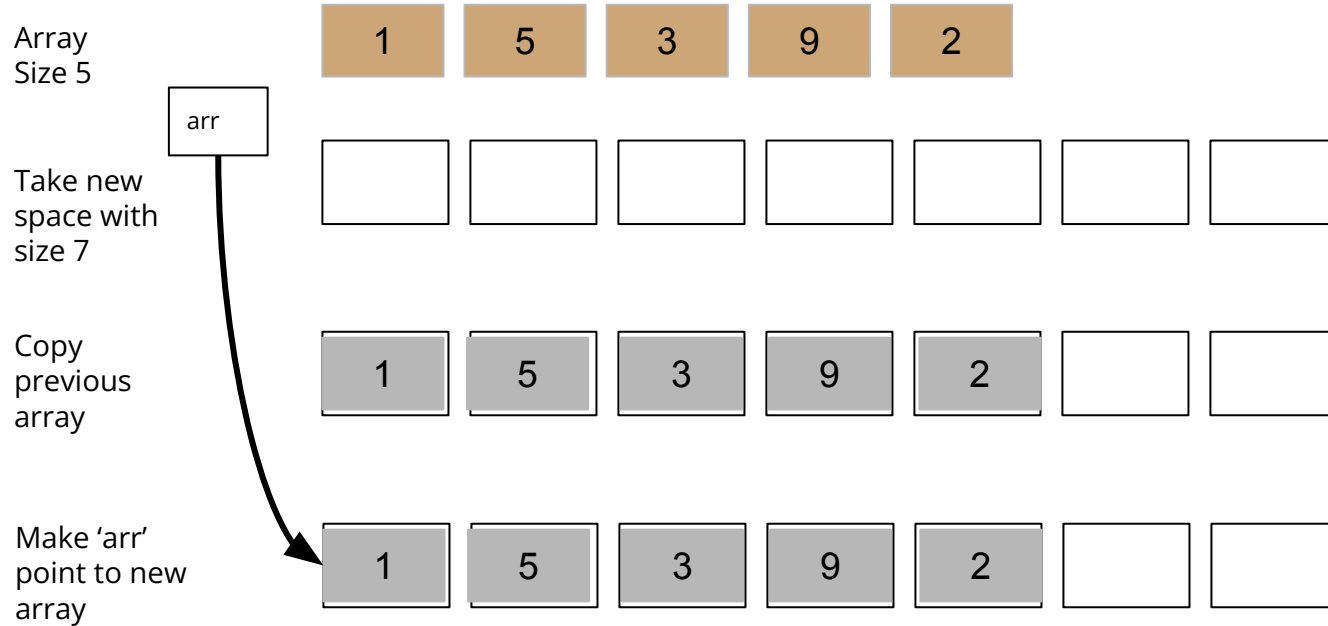
Take new
space with
size 7



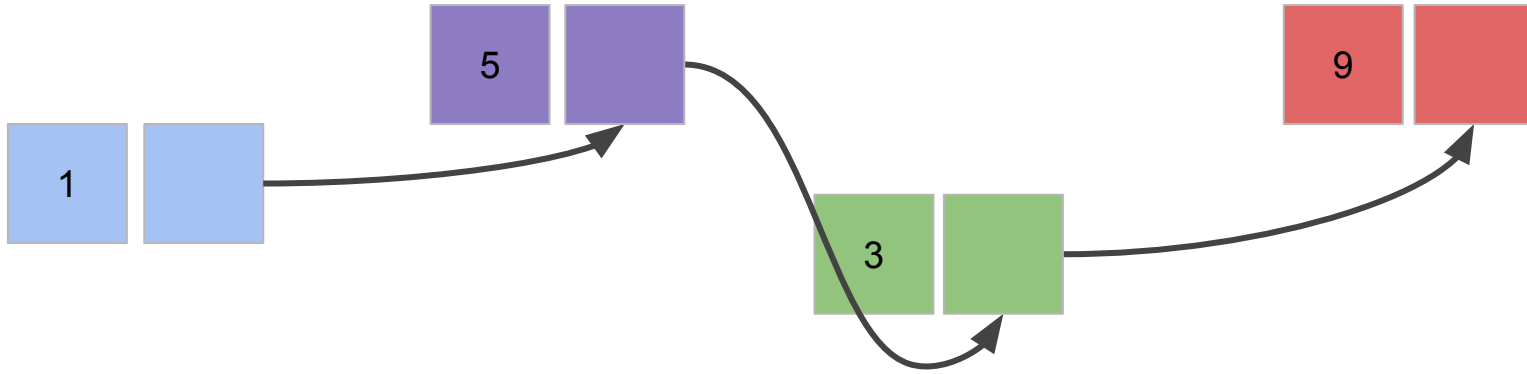
Copy
previous
array



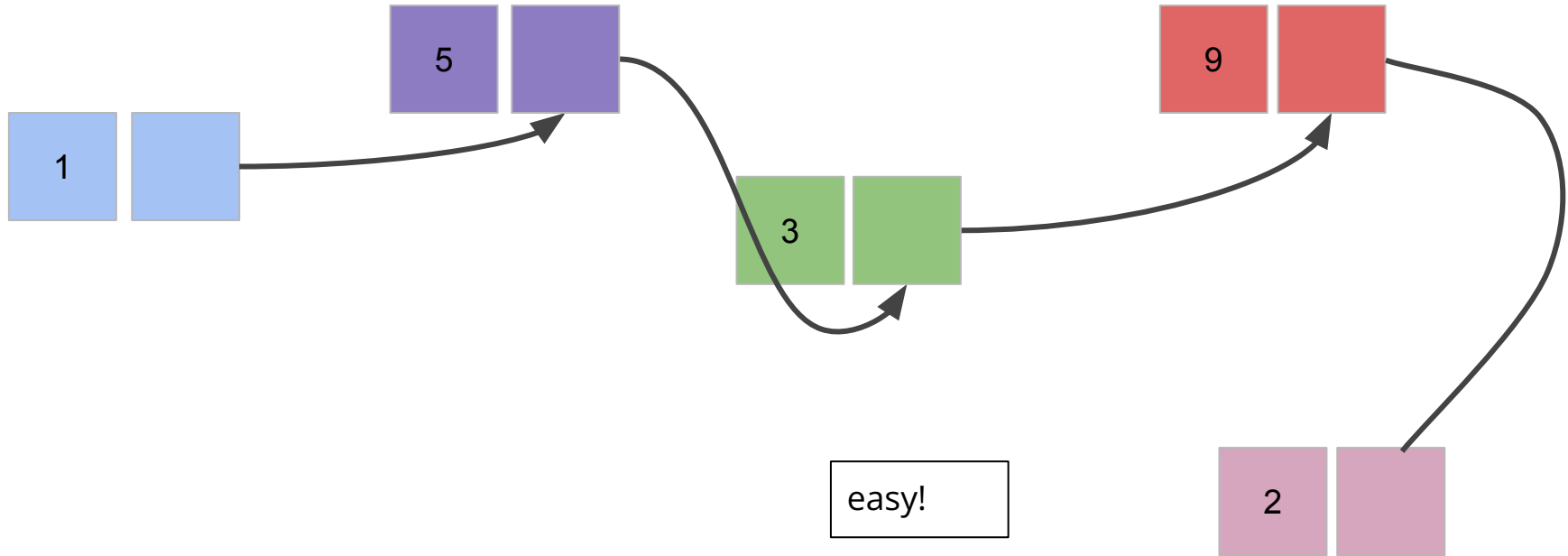
Resizing - Array



Resizing - Linked List

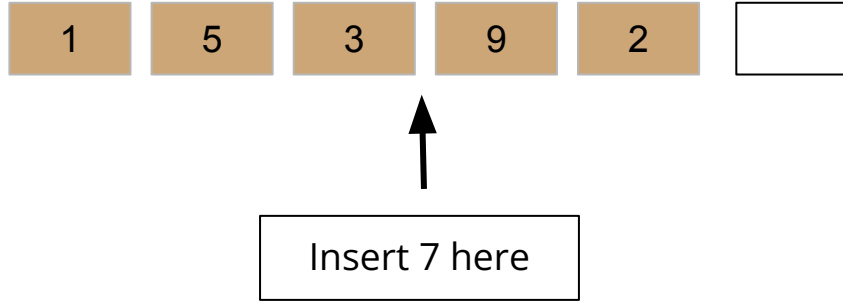


Resizing - Linked List



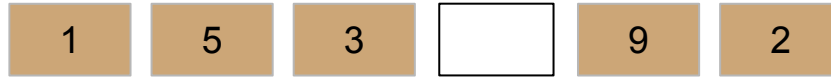
Insertion - Array

Array
Capacity 6
Size 5



Insertion - Array

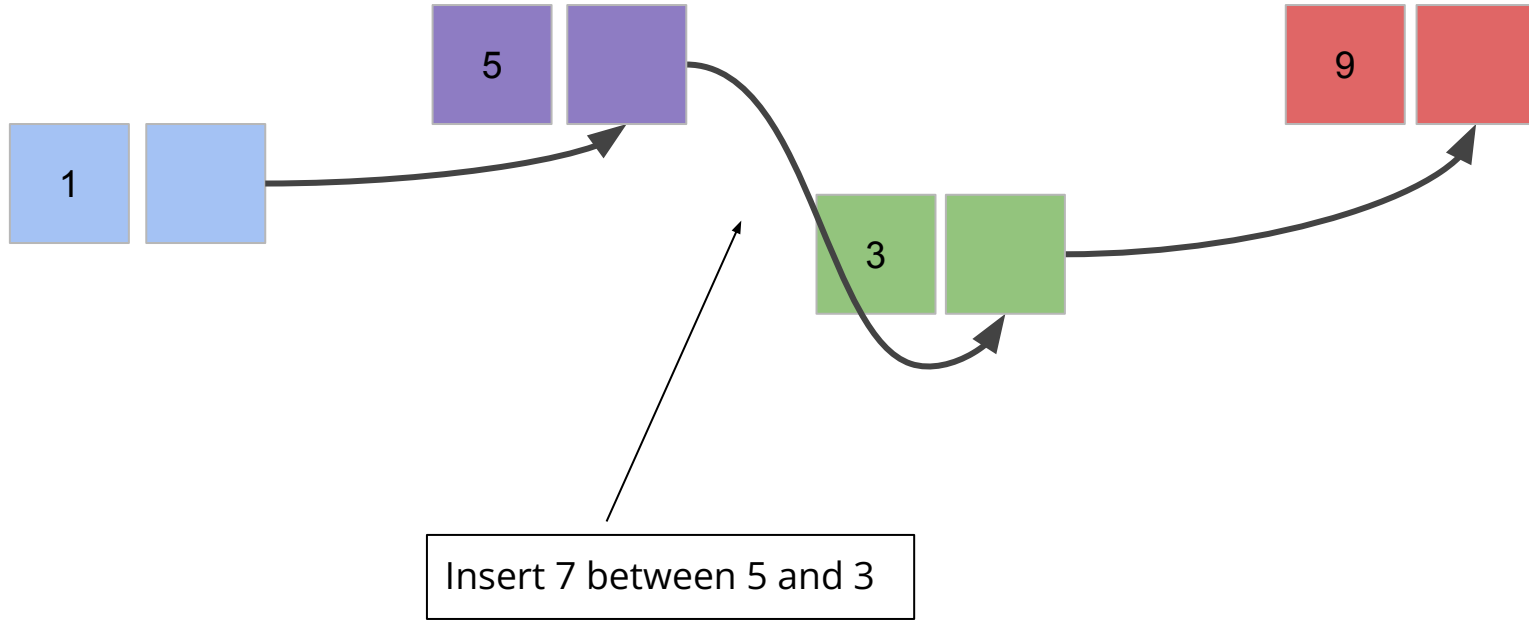
Array
Capacity 6
Size 5



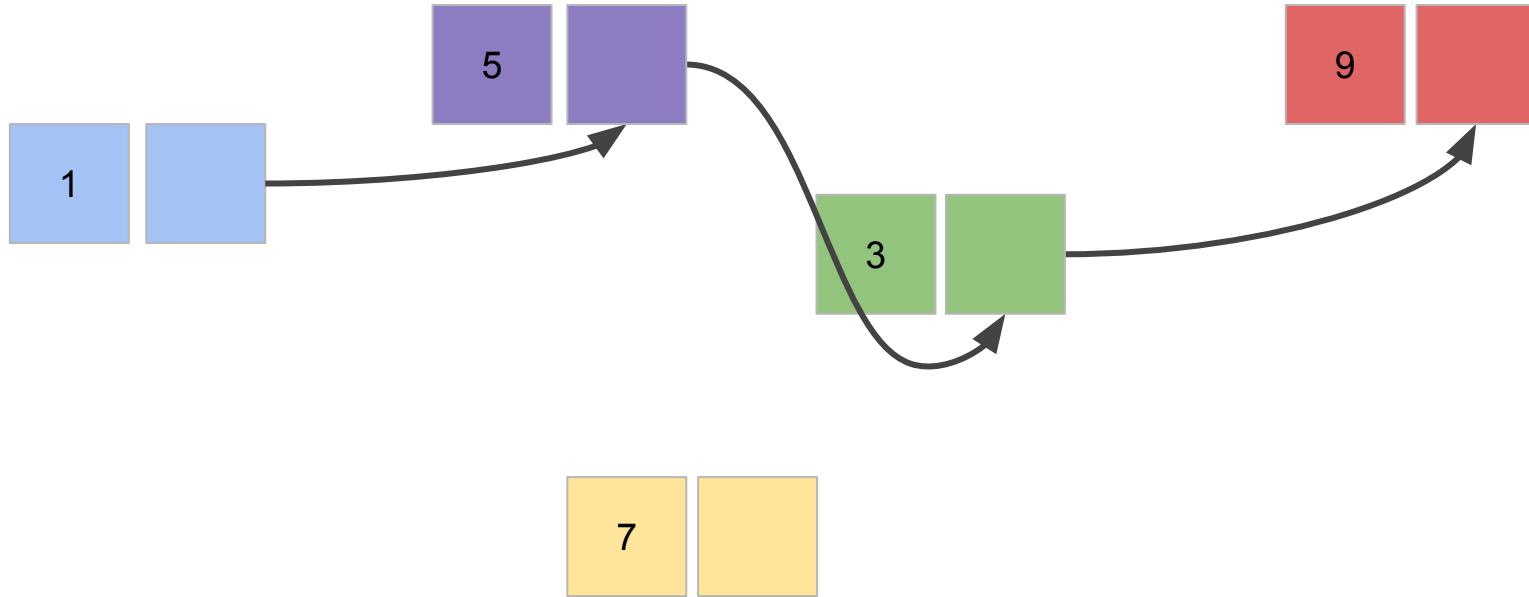
Insert 7 here

`array[3] = 7`

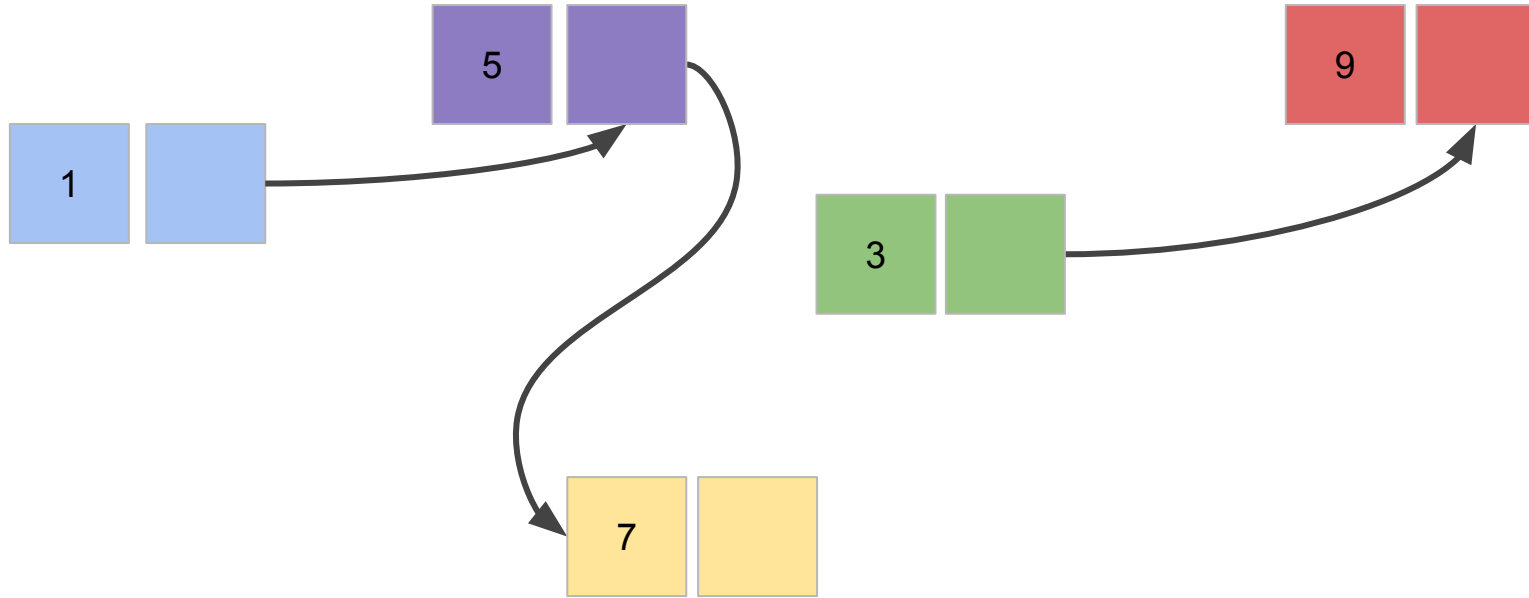
Insertion - Linked List



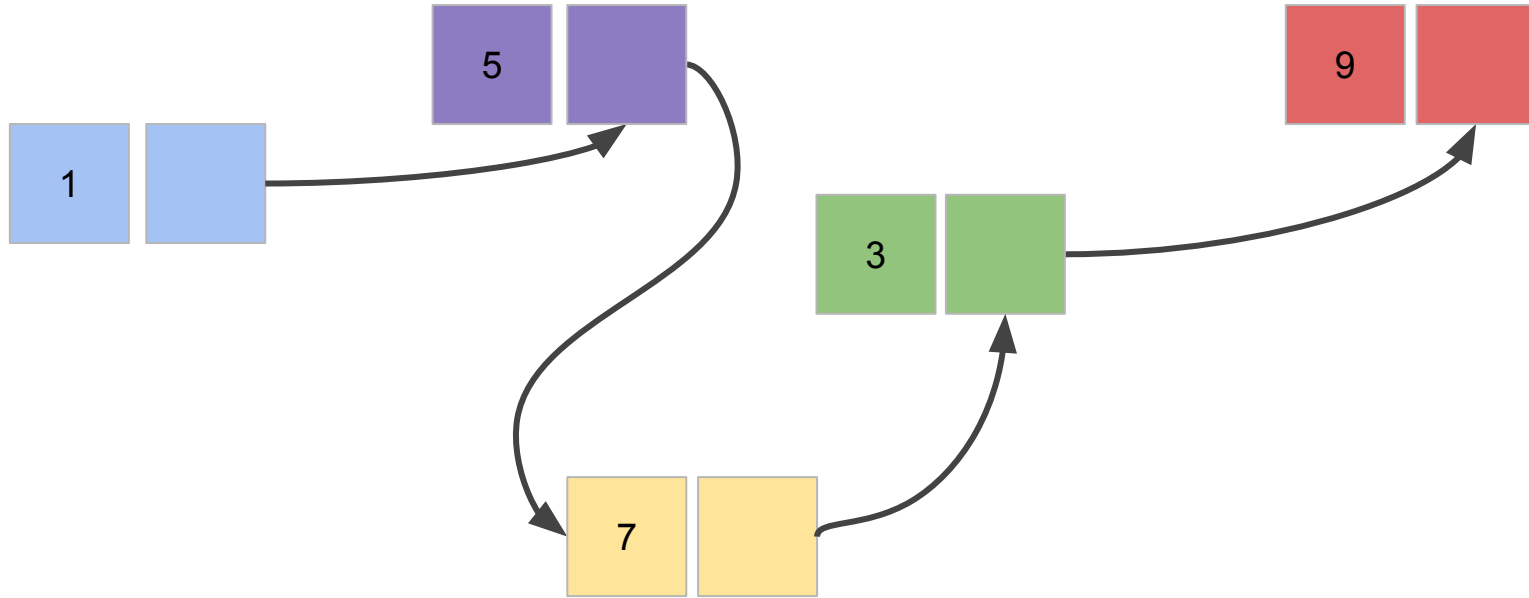
Insertion - Linked List



Insertion - Linked List



Insertion - Linked List



Removal - Array

Array



Delete 5

Removal - Array

Array



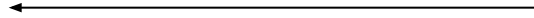
```
array[1] = None
```

Removal - Array

Array

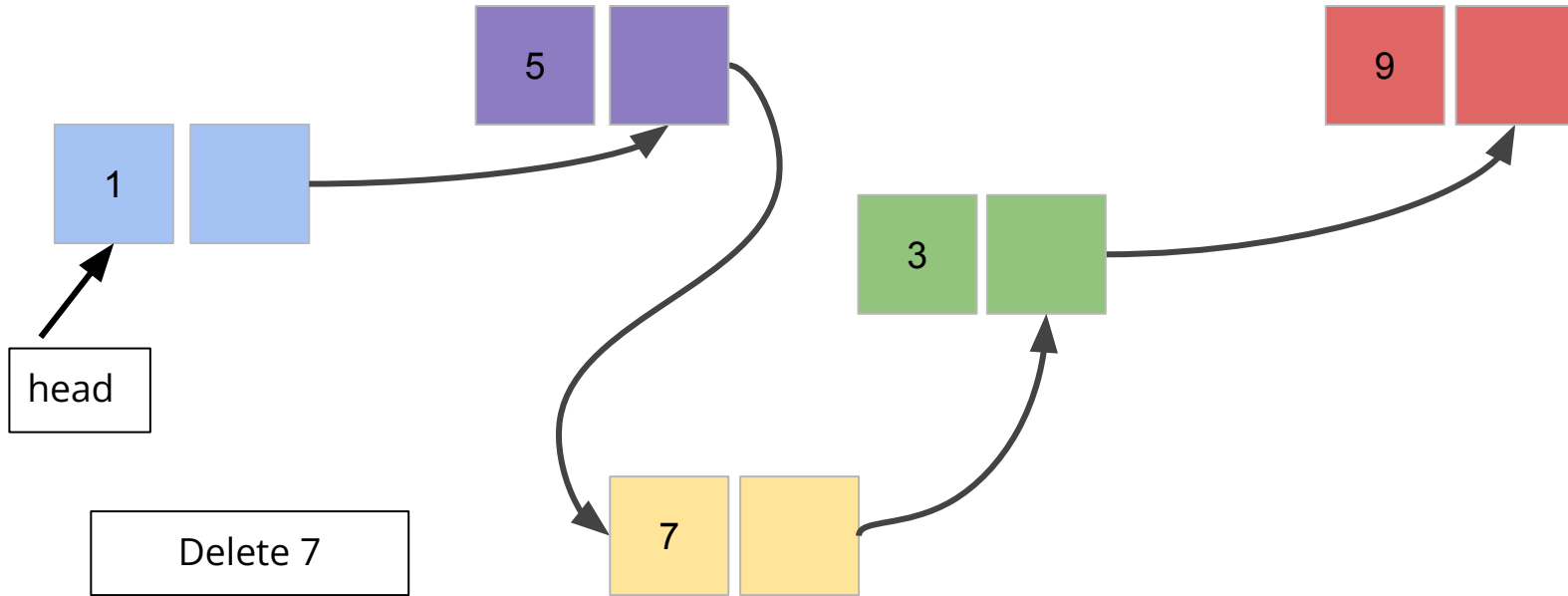


Shift left

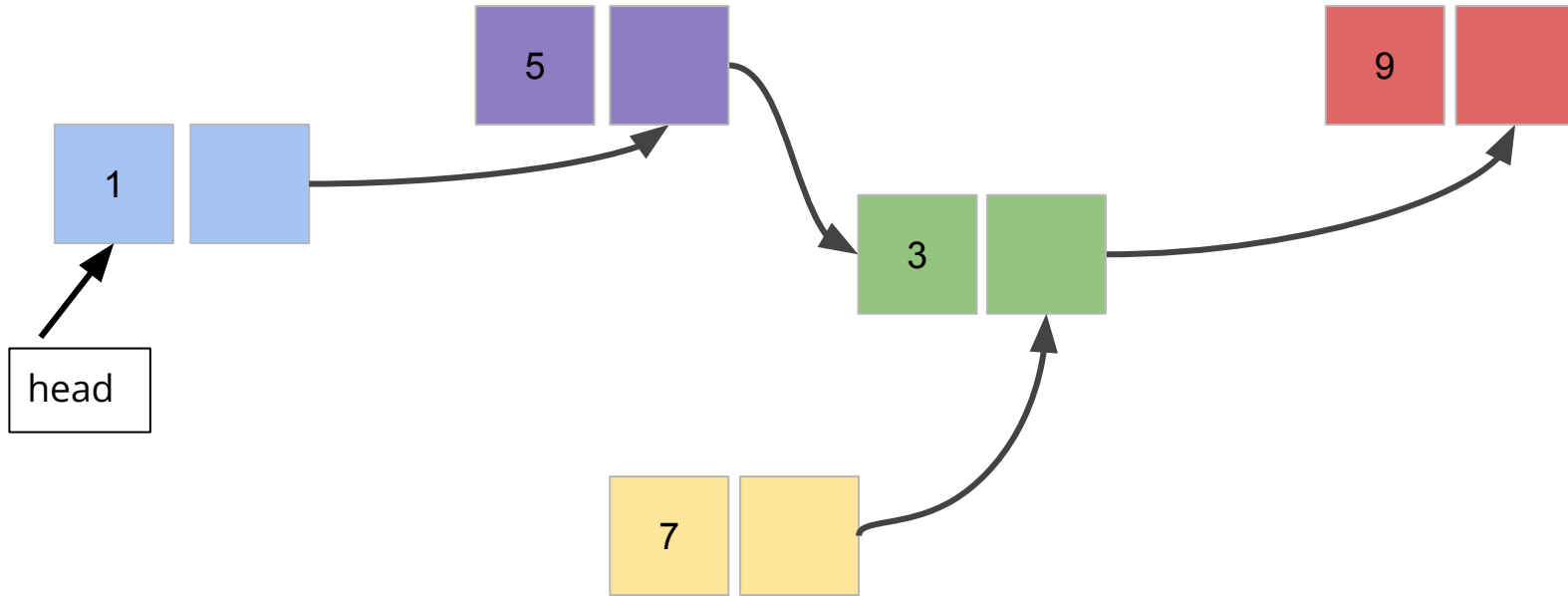


```
array[1] = None
```

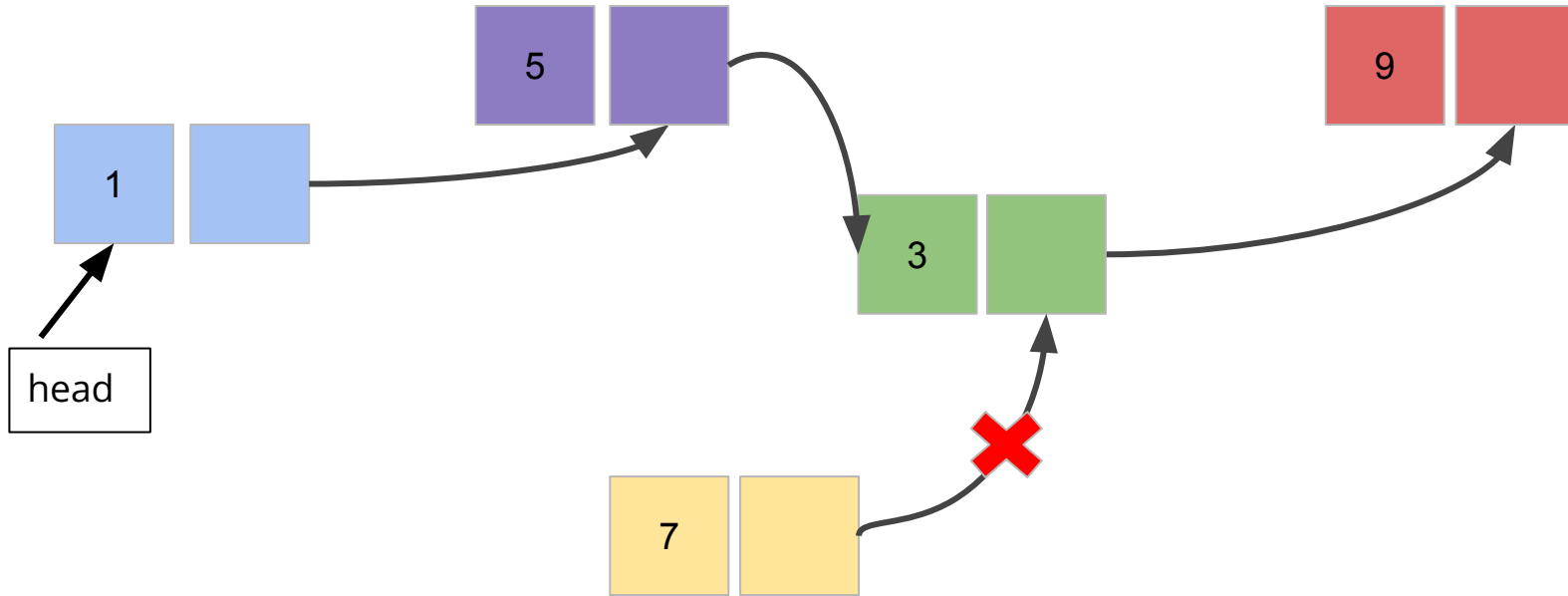
Removal - Linked List



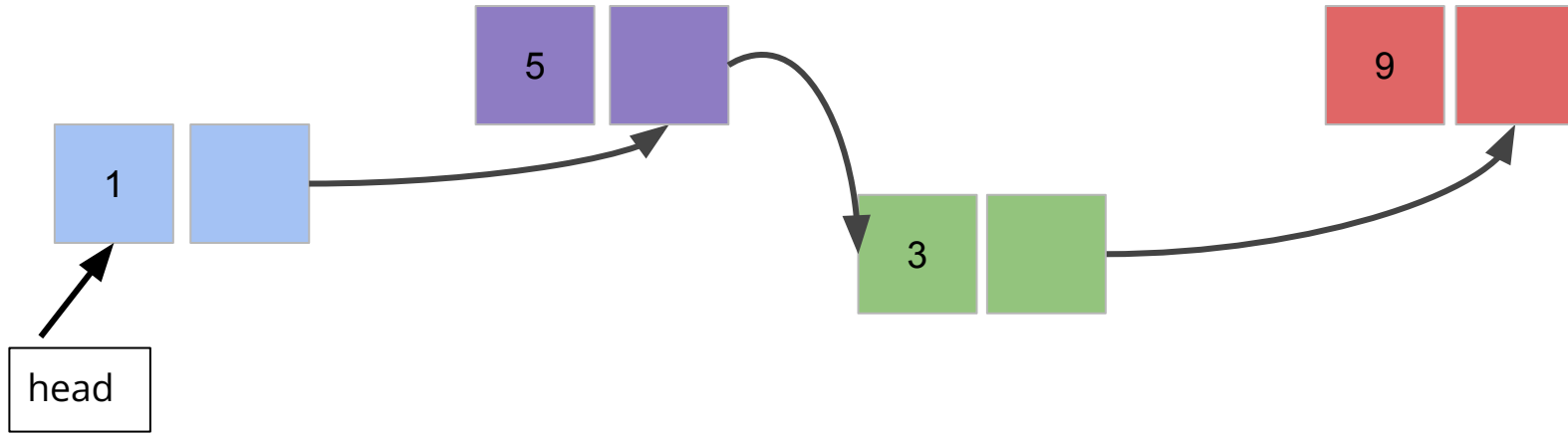
Removal - Linked List



Removal - Linked List

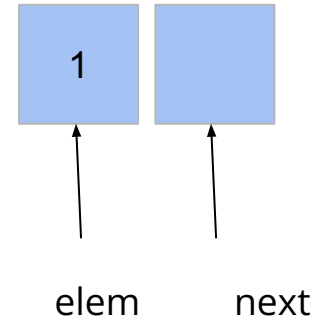


Removal - Linked List



Linked List Creation

```
1 # Node class design
2 class Node:
3     def __init__(self, e, n):
4         self.elem = e
5         self.next = n      (Address of next node)
6
```



Linked List Creation from an Array

```
1 # Creating a list
2 def createList(a):
3     head = Node(a[0], None)
4     tail = head
5     for i in range(1, len(a)):
6         n = Node(a[i], None)
7         tail.next = n
8         tail = tail.next
9     return head
```

Iterating Over a Linked List

```
1 # Iteration over a linked list
2 def iteration(head):
3     temp = head
4     while temp != None:
5         print(temp.elem    )
6         temp = temp.next
```

Counting elements of a Linked List

```
1 # Counting number of element in the list
2 def count(head):
3     count = 0
4     temp = head
5     while temp != None:
6         count += 1
7         temp = temp.next
8     return count
```

Retrieving an element from an index

```
1 # Getting element of an specific index
2 def elemAt(head, idx):
3     count = 0
4     temp = head
5     obj = None
6     while temp != None:
7         if count == idx:
8             obj = temp.elem
9             break
10        temp = temp.next
11        count += 1
12    if obj == None:
13        print("Invalid index")
14    return obj
```

Retrieving a node from an index

```
1 # Getting node of an specific index
2 def nodeAt(head, idx):
3     count = 0
4     temp = head
5     obj = None
6     while temp != None:
7         if count == idx:
8             obj = temp
9             break
10        temp = temp.next
11        count += 1
12    if obj == None:
13        print("Invalid index")
14    return obj
```

Update value of specific index

```
1 # Setting new element of an specific index
2 def set(head, idx, elem):
3     count = 0
4     temp = head
5     isUpdated = False
6     while temp != None:
7         if count == idx:
8             temp.elem = elem
9             isUpdated = True
10            break
11        temp = temp.next
12        count += 1
13    if isUpdated:
14        print("Value successfully updated!!!!")
15    else:
16        print("Invalid index")
```

Searching an element in the list

```
1 def contains(head, elem):  
2     temp = head  
3     while temp != None:  
4         if elem == temp.elem:  
5             return True  
6         temp = temp.next  
7     return False
```


Searching an element in the list

```
1 # Getting index of an specific element
2 def indexOf(head, elem):
3     temp = head
4     count = 0
5     while temp != None:
6         if elem == temp.elem:
7             return count
8         count += 1
9         temp = temp.next
10    return -1 # Here -1 represents the absence of element in the list
```